

ENSEMBLE METHODS (LEC 14)

Core Idea

Combine multiple classifiers to reduce generalization error. Can use same/different models, data, or hyperparameters.

Majority Voting:

$$\hat{y} = \text{mode}(C_1(x), C_2(x), \dots, C_m(x))$$

Weighted Voting (prob outputs):

$$\text{Score}_{\text{class}} = \sum_k w_k \cdot p_{k,\text{class}}$$

$$\hat{y} = \arg \max(\text{Score}_{\text{class}})$$

Bagging

- Train each classifier on a **randomly sampled subset with replacement**
- All classifiers trained independently (**parallel**)
- Reduces variance; different views of same data

AdaBoost

Weak learner = **decision stump** (1-node tree). Labels: -1 and $+1$. Sequentially learns from mistakes.

Init: $w_i = 1/n$ (uniform, $\sum w_i = 1$)
For $j = 1 \dots m$:
 $C_j = \text{train}(X, y, w)$
 $\hat{y} = \text{predict}(C_j, X)$
 $\varepsilon = w \cdot (\hat{y} \neq y)$ [weighted error]
 $\alpha_j = 0.5 \cdot \ln\left(\frac{1-\varepsilon}{\varepsilon}\right)$
 $w_i \leftarrow w_i \cdot e^{-\alpha_j \hat{y}_i y_i} \forall i$
 Normalize w to sum to 1
Final: $\hat{y} = \text{sign}\left(\sum_j \alpha_j \cdot \text{predict}(C_j, X)\right)$

Weight update logic:

- Correct: $\hat{y} \cdot y = +1 \Rightarrow e^{-\alpha} \Rightarrow$ weight **decreases**
- Wrong: $\hat{y} \cdot y = -1 \Rightarrow e^{+\alpha} \Rightarrow$ weight **increases**
- Large $\alpha_j \Rightarrow$ classifier trusted more in final vote

GRADIENT BOOSTING (LEC 15)

	AdaBoost	Grad. Boost
Weak learner	Stump	DT (3–4 lvls)
Learns from	Weighted err	Gradient of loss
Loss	Exp. loss	Logistic loss

$L_i = -[y_i \log p_i + (1 - y_i) \log(1 - p_i)]$
 $p = \text{sigmoid of predicted log-odds}$
Step 0: $\hat{y}_0 = \log(\#\text{pos}/\#\text{neg})$, $p_0 = \#\text{pos}/n$
Each round m :
 a) Residuals: $r_{im} = y_i - p_i$
 b) Fit new tree to $(X, \text{residuals})$
 c) Each leaf R_{jm} : $\gamma_{jm} = \frac{\sum (y_i - p_i)}{\sum p_i (1 - p_i)}$
 d) Update: $F_m(x) = F_{m-1}(x) + \eta \cdot \gamma_{jm}$
Inference: $\hat{y}_{\text{new}} = \hat{y}_0 + \eta(\gamma_{j1} + \gamma_{j2} + \dots + \gamma_{jm})$

Worked Example (2 pos, 1 neg):

$$\hat{y}_0 = \ln(2/1) = 0.69, \quad p_0 = 0.67$$

$$r(y=1) = 1 - 0.67 = 0.33; \quad r(y=0) = -0.67$$

$$\gamma_{\text{leaf}(y=1)} = \frac{0.33+0.33}{0.67 \times 0.33 \times 2} = 1.49$$

$$\gamma_{\text{leaf}(y=0)} = \frac{-0.67}{0.67 \times 0.33} = -3.12$$

Trees are **sequential**; stop when residuals $<$ threshold or M trees reached.

LINEAR REGRESSION – OLS (LEC 16)

Definition

Below is the definition of linear regression, the learning rules and model decision parameters

$$\hat{y} = w^\top x + b \quad (y \text{ is continuous})$$

$$\text{Loss (MSE): } L(w, b) = \frac{1}{n} \sum_i (y_i - \hat{y}^{(i)})^2$$

$$\frac{\partial L}{\partial w_j} = -\frac{2}{n} \sum_i (y_i - \hat{y}^{(i)}) x_j^{(i)}$$

$$\frac{\partial L}{\partial b} = -\frac{2}{n} \sum_i (y_i - \hat{y}^{(i)})$$

Regularization

Ridge (L2): $\text{MSE} + \lambda \|w\|_2^2$ (small, distributed w)

LASSO (L1): $\text{MSE} + \lambda \|w\|_1$ (sparse, some $w = 0$)

Elastic Net: $\text{MSE} + \lambda_1 \|w\|_1 + \lambda_2 \|w\|_2^2$

Residual Plots: plot $\varepsilon = y - \hat{y}$. Good: symmetric around 0. Bad: skewed/patterned.

Pearson Correlation (Feature Selection)

$$r = \frac{\sigma_{w_j y}}{\sigma_{w_j} \cdot \sigma_y}$$

$$\sigma_{w_j y} = \sum_i (x_j^{(i)} - \mu_j)(y^{(i)} - \mu_y)$$

$$\sigma_{w_j} = \sqrt{\sum_i (x_j^{(i)} - \mu_j)^2}$$

Range: -1 to $+1$; $|r| \approx 1 =$ strong linear relationship

Multi-collinearity: features are linearly dependent ($r \approx \pm 1$ between features). Fix: drop one.

Evaluation Metrics

$$\text{MSE} = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$$

$$\text{MAE} = \frac{1}{n} \sum |y_i - \hat{y}_i|$$

$$R^2 = 1 - \frac{\text{SSE}}{\text{SST}} = 1 - \frac{\text{MSE}}{\text{Var}(y)}$$

Train: $0 \leq R^2 \leq 1$; Test: R^2 can be **negative**

POLYNOMIAL & RANDOM FOREST REG (LEC 17)

Polynomial Regression

Add polynomial features, then train OLS. Degree d : add all products up to degree d .

4 features, degree 2 $\Rightarrow$ 15 total features:
 $1, x_1, x_2, x_3, x_4, x_1^2, x_2^2, x_3^2, x_4^2,$
 $x_1 x_2, x_1 x_3, x_1 x_4, x_2 x_3, x_2 x_4, x_3 x_4$

Decision Tree Regression

Same greedy split but use **MSE**; node prediction = **mean** of samples.

$$I(D) = \frac{1}{N_D} \sum_{i \in D} (y_i - \bar{y}_D)^2$$

$$\text{Gain}(x_i) = I(D) - \frac{N_i}{N} I(D_L) - \frac{N_r}{N} I(D_r)$$

Random Forest Regression

Ensemble of DT Regressors using **bagging**; final prediction = **average** of all trees.

Model	Linear?	Loss
OLS	Yes	MSE
Ridge	Yes	$\text{MSE} + \lambda \ w\ _2^2$
LASSO	Yes	$\text{MSE} + \lambda \ w\ _1$
Polynomial	No	MSE on poly feats
Rand. Forest	No	MSE per node

	Bagging	Boosting
Training	Parallel	Sequential
Sampling	Random w/ replace.	Weighted
Learners	Independent	Each fixes prev.
Reduces	Variance	Bias

K-MEANS CLUSTERING (LEC 17–18)

Unsupervised; no labels y . Find K natural clusters minimizing within-cluster SSE.

$$D = \sum_i \sum_k r_{ik} \|x^{(i)} - \mu_k\|^2$$

$$r_{ik} = 1 \text{ if sample } i \text{ in cluster } k, \text{ else } 0$$

$$\mu_k = \frac{\sum_i r_{ik} x^{(i)}}{\sum_i r_{ik}} \quad [\text{cluster mean}]$$

Algorithm:

- Pick K random points as initial μ_k
- Assign each sample to nearest centroid
- Update $\mu_k =$ mean of assigned samples
- Repeat until convergence

Elbow Method (choosing K): Run K-means for $k = 1 \dots K'$; plot D vs k ; choose k at the “elbow.” $D = 0$ is trivial ($K=N$).

Silhouette Value

$a^{(i)}$ = avg dist to points in *same* cluster
 $b^{(i)}$ = avg dist to points in *nearest other* cluster
 $s^{(i)} = \frac{b-a}{\max(b,a)}$
 $s \approx +1$: well-matched; $s \approx 0$: boundary; $s \approx -1$: wrong cluster

HIERARCHICAL CLUSTERING (LEC 19)

No need to choose K upfront; builds a **dendrogram**.

Agglomerative (bottom-up):

- Start: N clusters (one per sample)
- Compute pairwise distances
- Merge closest pair
- Repeat until 1 cluster

Single linkage: $\min d(u, v)$
 Complete linkage: $\max d(u, v)$

Cut dendrogram at desired level to extract K clusters. Application: phylogenetic trees.

NEURAL NETWORKS (LEC 19–21)

Architecture

Input Layer $\rightarrow$ Hidden Layers $\rightarrow$ Output Layer.
 Each neuron: $a = \sigma(w^\top x + b)$.

Counting Parameters:

Weights = $\sum(\text{dim}_{\text{prev}} \times \text{dim}_{\text{next}})$
E.g. input=2, hidden=[3,3], output=1:
 Weights = $2 \times 3 + 3 \times 3 + 3 \times 1 = 18$; Neurons = 9
 Weight label: w_{ij} = from neuron i to neuron j

Activation Functions

Sigmoid: $\sigma(z) = \frac{1}{1+e^{-z}}$; $\frac{d\sigma}{dz} = \sigma(z)(1-\sigma(z))$
 ReLU: $\max(0, z)$ [preferred in deep nets]
 Step function: $\mathbf{1}[z > 0]$ [Perceptron]

Forward Propagation

Layer by layer, left $\rightarrow$ right: $a_n = \sigma(\sum w_{\text{prev},n} \cdot a_{\text{prev}} + b_n)$

Example ($2 \rightarrow [3] \rightarrow 1$, all $w = 1$, $b = 0$):

$$x = (0, 0): a_3 = \sigma(0) = 0.5, \quad a_5 = \sigma(1.0) = 0.73$$

$$x = (0, 1): a_3 = 0.73, \quad a_5 = \sigma(1.46) = 0.81$$

$$x = (1, 1): a_3 = 0.73, \quad a_5 = \sigma(1.46) = 0.85$$

Loss (MSE): $L = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$; $\frac{dL}{da_5} = -2(y - a_5)$

Backpropagation (Chain Rule)

Compute **right-to-left**; reuse intermediate gradients.

$$\frac{dL}{dw_{35}} = \frac{dL}{da_5} \cdot \frac{da_5}{dw_{35}}$$

$$\frac{da_5}{dw_{35}} = a_5(1 - a_5) \cdot a_3$$

$$\frac{dL}{dw_{13}} = \frac{dL}{da_5} \cdot \frac{da_5}{da_3} \cdot \frac{da_3}{dw_{13}}$$

$$\frac{da_5}{da_3} = a_5(1 - a_5) \cdot w_{35}; \quad \frac{da_3}{dw_{13}} = a_3(1 - a_3) \cdot x_1$$

Multi-path (neuron 3 $\rightarrow$ 6,7,8): sum gradients over all paths

Training Loop:

- Init weights (small random)
- Forward prop $\rightarrow$ compute $\hat{y}$

3. Compute loss $L(\hat{y}, y)$
4. Backward prop $\rightarrow$ gradients
5. Update: $w \leftarrow w - \eta \cdot \frac{dL}{dw}$
6. Repeat until convergence

DEEP LEARNING (LEC 21)

ANN with > 3 layers = Deep Learning. Sigmoid in hidden layers $\rightarrow$ **vanishing gradients**; fix with **ReLU** in hidden layers.

- More layers = more abstract feature representations
- Deeper nets require careful weight initialization to train stably
- Use **cross-entropy loss** for multi-class classification
- Optimization via **SGD** (stochastic gradient descent) or variants (Adam, etc.)
- Training mode: gradients enabled; Eval mode: gradients disabled

QUICK FACTS & EXAM TRAPS**Ensemble**

- AdaBoost labels are -1 and $+1$, **NOT** 0 and 1
- Sample weights $\neq$ model weights (as in Adaline)
- ϵ = weighted error; α_j larger when ϵ small (better classifier)
- Correct pred: weight $\downarrow$; Wrong pred: weight $\uparrow$
- Bagging = parallel; Boosting = sequential
- Gradient boosting residual $r_i = y_i - p_i$ (not $\hat{y}$)
- $\gamma_{j\text{m}} = \sum r / \sum p(1 - p)$ for each leaf
- Inference: sum all γ contributions $\times \eta$

Regression

- OLS = Adaline but y continuous (no threshold)
- Pearson r measures **linear** correlation only
- R^2 on test can be **negative** (worse than mean predictor)
- $R^2=1$ = perfect fit; $R^2=0$ = always predicts

 μ_y

- LASSO $\rightarrow$ sparse weights (feature selection)
- Ridge $\rightarrow$ small distributed weights
- DT Regression: node prediction = **mean**; split by MSE gain
- Random Forest Regression: bagging + MSE splits

Clustering & Neural Networks

- K-means: $D = 0$ requires $K = N$ (trivial/useless)
- Silhouette: $+1$ =good, 0 =boundary, -1 =wrong cluster
- Hierarchical: no need to pick K first; cut dendrogram
- Single linkage = min dist; Complete = max dist
- NN weights per layer pair = $d_{\text{prev}} \times d_{\text{next}}$
- Backprop: **right-to-left**; reuse intermediate gradients
- Multiple paths $\rightarrow$ **sum** gradients from all paths
- $\frac{d\sigma}{dz} = \sigma(z)(1 - \sigma(z))$ $\leftarrow$ memorize!
- ReLU fixes vanishing gradient in deep nets